CODERHOUSE · DATA ENGINEERING

Arquitectura y Auditoría Técnica

Capstone Real-Time Data Platform

Nicolás López · 7 de noviembre de 2026

PRELIMINAR

Pendiente de despliegue limpio y prueba simultánea en AWS.

Validaciones locales verificadas. Evidencias PE5/PE6 identificadas como antecedentes.

Repositorio GitHub

Contenido

1. Portada 3
2. Resumen ejecutivo 3
3. Objetivo del proyecto 3
4. Alcance de la plataforma 3
5. Arquitectura general 4
6. Diagrama end-to-end 4
7. Inventario de servicios AWS 5
8. Estructura del repositorio 5
9. Infraestructura como código 6
10. Parámetros críticos de configuración 6
11. Pipeline de ingesta 7
12. Procesamiento con Flink 7
13. Event Time, watermarks y ventanas 7
14. Checkpoints y recuperación de estado 8
15. Lakehouse con Iceberg y Glue 8
16. Redshift Streaming Ingestion 8
17. Materialized Views y consultas SQL 9
18. Consistencia de datos y reconciliación 9
19. Análisis de backpressure 9
20. Manejo de fallos 10
21. Observabilidad y monitoreo 11
22. Seguridad e IAM 12
23. Optimización y costos 12
24. Pruebas end-to-end 13
25. Evidencias reales 13
26. Análisis de trade-offs 18
27. Limitaciones y riesgos residuales 18
28. Conclusiones 18
29. Referencias técnicas 18
30. Enlace al repositorio GitHub 19

1. Portada

Documento de Arquitectura y Auditoría Técnica (DAAT)

Nicolás López · Coderhouse · Data Engineering · Proyecto Final

Fecha de revisión: 7 de noviembre de 2026. Región de referencia: us-east-1. Rama: feature/final-capstone-audit. Base auditada: 9254d9a, entrega 6.

PRELIMINAR — pendiente de despliegue limpio y validación simultánea en AWS. Este documento no debe presentarse como certificación de una prueba que todavía no se ejecutó. No se hicieron apply, reinicios ni eliminaciones AWS para esta auditoría.

2. Resumen ejecutivo

El repositorio implementa una plataforma de transporte y consulta con dos rutas: Flink transforma eventos Kinesis en agregaciones Iceberg registradas en Glue; Redshift consume el mismo transporte directamente mediante Streaming Ingestion. PE5 verificó persistencia transaccional y consulta Athena. PE6 verificó ingesta, validación JSON, consultas y seguridad SQL contra el Lakehouse histórico. Esos ensayos son reales, pero no equivalen a desplegar todo desde cero ni a reconciliar dos consumidores activos sobre un nuevo lote común.

La auditoría corrige el promedio decimal en SQL, la tolerancia a campos JSON adicionales y la visibilidad de eventos tardíos. Se consolidaron Terraform, aplicación y SQL en las carpetas exigidas, con un camino aislado que carga el JAR e inicializa SQL. Diez pruebas Java y cinco Python pasan localmente. El plan de dev no propone cambios en recursos; solo un output. La prueba limpia y sus capturas siguen pendientes de autorización y ejecución.

3. Objetivo del proyecto

Transportar transacciones, calcular métricas aprobadas por minuto y método de pago, conservarlas en un Lakehouse con commits atómicos y ofrecer consulta de datos recientes e históricos. El evaluador debe poder reproducir la plataforma, interpretar sus garantías reales y distinguir evidencia observada de una predicción arquitectónica. El criterio de aprobación final incluye un entorno nuevo, SQL inicializado y métricas conciliadas después del cierre y commit de ventanas.

4. Alcance de la plataforma

Se auditaron los seis antecedentes, módulos Terraform, Java/Maven/pruebas, scripts SQL, procedimientos de operación y evidencias existentes. El inventario inicial registra 294 archivos rastreados y sus SHA256. El escaneo de patrones no encontró claves AWS con formato conocido ni encabezados de clave privada; esto no constituye una garantía exhaustiva sobre todo el historial Git.

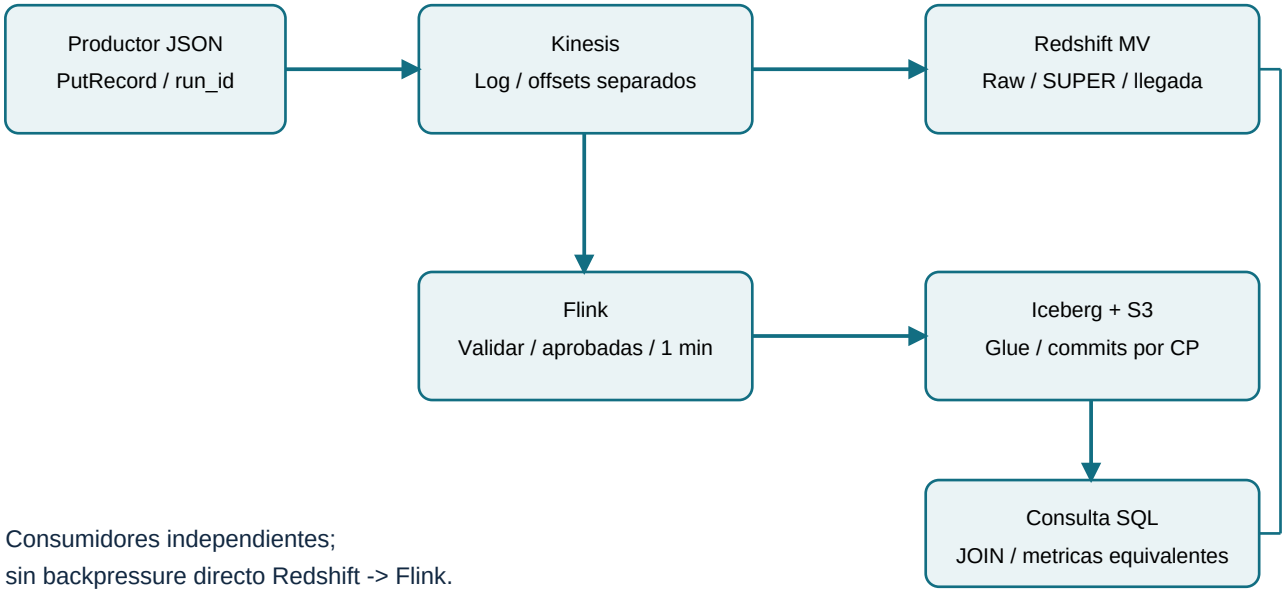
El laboratorio Kafka/Spark de PE3 es independiente y local. No se presenta como un componente conectado artificialmente al flujo AWS. Sus límites académicos incluyen broker único y almacenamiento efímero. Quedan fuera de esta ejecución el ensayo de carga, caos AWS, recuperación multirregión y datos personales reales.

5. Arquitectura general

El productor usa PutRecord con JSON y partition key. Kinesis retiene el log; Flink y Redshift mantienen posiciones independientes. Flink valida, asigna Event Time y acumula aprobadas por método en ventanas de un minuto. Iceberg escribe Parquet en S3 y publica metadatos mediante GlueCatalog. Redshift guarda la ingesta raw en una MV, valida campos mediante SUPER/TRY_CAST y consulta Iceberg a través de un esquema externo Glue.

La VPC tiene dos subredes privadas y endpoint gateway S3, sin NAT ni Internet Gateway. Redshift no es público; un endpoint interface Kinesis permite su lectura privada. Flink Managed no está asociado a esta VPC y accede a servicios AWS mediante su red administrada. No atribuirle el aislamiento de subred de Redshift.

6. Diagrama end-to-end



Consumidores independientes;
sin backpressure directo Redshift -> Flink.

El diagrama del PDF distingue transporte, transformaciones, almacenamiento y consulta. Los dos consumidores comparten capacidad de lectura del stream pero no una cadena de operadores. Una consulta JOIN no inserta registros en Iceberg. La reconciliación espera visibilidad en ambas rutas y usa agregados equivalentes; no compara una fila raw con una fila resumen como si tuvieran igual granularidad.

7. Inventario de servicios AWS

Componente	Estado observado al auditar	Fuente
Kinesis dev	ACTIVE, dos shards, retención 24 h, KMS	initial-aws.json
Flink dev	READY, detenido; versión desplegada anterior	DescribeApplication
S3 Lakehouse dev	Versionado, AES256, bloqueo público	APIs S3
Glue dev	Base lakehouse_db y tabla transaction_aggregates	GetTable
Redshift temporal PE6	Sin workgroups ni namespaces al auditar	List APIs
Red/VPC/IAM	Recursos existentes conservados	Estado y plan Terraform
Entorno capstone	No desplegado	No hay recibo de apply nuevo

Los nombres e identificadores históricos son evidencia, no valores requeridos para crear el nuevo entorno. `initial-aws.json` conserva el detalle de diez consultas satisfactorias; los archivos de plan documentan recursos y direcciones.

8. Estructura del repositorio

Ruta	Responsabilidad
terraform/	Plataforma modular y backend parcial
terraform/bootstrap/	Backend nuevo, separado del existente
terraform/isolated/	Raíz capstone con nombres y estado únicos
flink-app/	Java17, Flink2.2.1, Iceberg1.12.0, JAR1.2.0
sql/redshift/	SQL numerado y consolidado
scripts/	Auditoría, pruebas, inicialización, PDF
docs/evidence/	Evidencias históricas y validaciones actuales
output/pdf/	DAAT y PDF histórico de PE6

`infra/` y `app/flink-transactions/` conservan notas de compatibilidad. Se actualizaron referencias, sin duplicar la implementación. Los tres JSON locales previos de `applications/` no se incorporan al entregable. Estados, cachés, variables locales, artefactos Maven y manifiestos temporales están ignorados.

9. Infraestructura como código

Módulos: network, identity, kinesis, managed-flink, lakehouse y redshift. La cuenta se obtiene con aws_caller_identity. Región, entorno, perfil opcional, bucket, base Glue y artefacto son parámetros. El backend S3 es parcial; se completa con un archivo local ignorado. Dev conserva bucket/clave/locks originales. El nuevo backend usa versionado y locking nativo S3; requiere Terraform1.10 o superior.

El JAR se publica como aws_s3_object con hash y object_version, referenciado por Flink. La raíz aislada arranca Flink e inicializa SQL mediante terraform_data y Data API. El script espera la tabla Iceberg antes de crear las vistas externas. AWS CLI/Python/Maven siguen siendo prerrequisitos explícitos del ejecutor.

```
artifact_key                = "artifacts/flink/flink-transactions-1.2.0.jar"
start_flink                 = true
flink_snapshots_enabled     = true
harden_flink_trust          = true
initialize_platform         = true
```

Las tres raíces validan y fmt pasa. El plan refrescado de dev agrega solo el output flink; no hay altas, modificaciones ni bajas de recursos. Esto verifica compatibilidad con el estado actual, **no reproducibilidad demostrada desde cero**. El runbook documenta bootstrap, plan nuevo, apply autorizado y retiro aislado. Además se generó una previsualización desde estado local vacío: 36 altas, 0 cambios y 0 bajas, más 4 altas del bootstrap. Revisa el grafo sin apply; el despliegue real usará un backend S3 nuevo.

10. Parámetros críticos de configuración

Parámetro	Configuración	Efecto
Kinesis	PROVISIONED, 2 shards, 24 h	Capacidad/ventana de recuperación
Posición inicial nueva	TRIM_HORIZON	Lee el log retenido si no restaura offsets
Flink	Paralelismo1, 1 por KPU, sin autoescalado	Coste bajo; capacidad limitada
Checkpoint	60000 ms, pausa mínima5000 ms, uno simultáneo	Publicación y recuperación
Watermark	Máximo observado menos10 s	Tolera desorden acotado
Idleness	30 s en asignación posterior a fuente	No es seguimiento individual por shard
Ventana	Tumbling Event Time de1 min UTC	Una agregación por método/minuto
Iceberg	v2, Parquet/Snappy, objetivo128 MiB	Archivos pequeños con baja carga
Partición	days(window_start)	Poda temporal sin carpetas por minuto
Redshift	Base/máximo4 RPU, privado	Consulta/ingesta acotadas, costo activo

11. Pipeline de ingesta

El contrato exige transaction_id, timestamp, amount, payment_method y status. La prueba nueva agrega produced_at como dato del productor. La hora aproximada de llegada la suministra Kinesis y Redshift la conserva como approximate_arrival_timestamp/arrival_timestamp; no sustituye Event Time. Los metadatos adicionales ya no invalidan el JSON en Flink.

El fixture tiene diez registros, un run_id único y una misma partition key para controlar el orden de llegada. No pretende medir distribución entre shards ni throughput. PutRecord devuelve shard y sequence number, guardados con hora UTC. Una respuesta perdida puede hacer ambiguo un reintento del productor; no existe idempotencia de negocio basada en transaction_id. El guard de publicación evita repetir automáticamente una prueba parcialmente enviada.

12. Procesamiento con Flink

TransactionParser valida tipos, importe no negativo y escala monetaria, campos requeridos y timestamp. Los inválidos se descartan en Flink y se cuentan; Redshift preserva raw y los clasifica. El estado se normaliza y solo approved alimenta la agregación. BigDecimal evita aritmética binaria para importes. El promedio Java se calcula a seis decimales.

TransactionPipeline centraliza validación y agregación para usar el mismo código en job y pruebas. Contadores: valid_records, invalid_records y late_records. El side output tardío registra SHA256 del identificador, sin monto ni ID raw. Los duplicados de entrada se cuentan: un contador válido no demuestra unicidad. Los UIDs estables nuevos se destinan al despliegue limpio; no se probó migración de estado desde el JAR1.1 desplegado anteriormente.

13. Event Time, watermarks y ventanas

El timestamp proviene del evento. El watermark se calcula después de la fuente y validación; con paralelismo1 observa los registros ya mezclados. No se debe describir idleness de30 s como un mecanismo por shard en esta implementación. Con más subtareas, el watermark del operador depende de entradas activas.

Una ventana de un minuto cierra cuando el watermark alcanza su final; no por el reloj de pared. La tolerancia de desorden de10 s no implica aceptar cualquier evento durante10 s de tiempo real. allowedLateness es cero: si una ventana ya fue eliminada, la aprobada tardía va al side output y no corrige su agregado. Un evento anterior al watermark todavía puede pertenecer a una ventana abierta.

```
WatermarkStrategy.<TransactionEvent>forBoundedOutOfOrderness(Duration.ofSeconds(10))
    .withTimestampAssigner((event, previous) -> event.getEventTimeMillis())
    .withIdleness(Duration.ofSeconds(30));
```

La prueba local emite watermarks determinísticos para verificar el límite de cierre. La prueba AWS preparada usa eventos rechazados válidos para avanzar el tiempo sin alterar sumas aprobadas y espera el commit antes de enviar el tardío.

14. Checkpoints y recuperación de estado

El checkpoint coordina posiciones de fuente, estado de ventanas y sink. El modo EXACTLY_ONCE está explicitado y el sink Iceberg publica archivos pendientes al confirmarse checkpoints. Su atomicidad no elimina duplicados de negocio enviados dos veces al stream ni crea una transacción distribuida con Redshift.

Dos pruebas locales del sink pasan. Una verifica commits/snapshots; otra provoca un fallo tras checkpoints, reinicia el job y verifica total de20 registros y múltiples snapshots. Esta última comprueba cantidad final; no certifica por sí sola la identidad de cada registro ni recuperación del servicio administrado. Los avisos de fallo deliberado y shutdown en el log pertenecen a esa prueba.

Para AWS: registrar snapshot compatible y versión, restaurar explícitamente, comprobar offsets, ventanas y commits. Si se inicia sin estado sobre una tabla existente con TRIM_HORIZON, pueden duplicarse agregaciones ya publicadas. El runbook exige un entorno nuevo y no cambia la aplicación dev para demostrar esto.

15. Lakehouse con Iceberg y Glue

Terraform crea la base Glue; la aplicación crea o valida la tabla mediante org.apache.iceberg.aws.glue.GlueCatalog y S3FileIO. Iceberg controla su puntero metadata_location; una tabla Glue genérica no reemplaza esa inicialización. El esquema contiene inicio/fin de ventana, método, cantidad, total decimal(38,2) y promedio decimal(38,6). Datos Parquet y JSON de metadatos residen bajo el prefijo de la tabla. El versionado S3 protege versiones de objetos, no sustituye snapshots Iceberg ni una estrategia de backup probada.

Se elige days(window_start) para permitir partition pruning por rango temporal sin multiplicar particiones por cada minuto o método. El filtro de consultas debe conservar una restricción útil sobre window_start. Un objetivo de128 MiB no evita archivos pequeños cuando se confirma cada minuto con pocos eventos. La compactación, expiración de snapshots y eliminación de huérfanos requieren políticas adicionales que respeten lectores y retención; no se ejecutaron.

PE5 verificó snapshot8171898520026477263, checkpoint2 y consulta Athena con 2 aprobadas, total30.30 y promedio15.150000. Es un antecedente verificable, no el resultado del fixture nuevo. La base conserva prevent_destroy; el retiro capstone exige comprobar propiedad y borrar solo su tabla/base antes de destruir el estado temporal, como se explica en el runbook.

16. Redshift Streaming Ingestion

El esquema externo kinesis_source usa IAM_ROLE default y apunta a Kinesis. La MV streaming.transactions_raw conserva coordenadas shard/sequence, llegada, hora de refresh, bandera JSON y payload SUPER. CAN_JSON_PARSE evita que JSON inválido aborte el parseo. Las vistas de conversión usan TRY_CAST y un motivo de rechazo.

AUTO REFRESH YES está declarado; las pruebas controladas ejecutan REFRESH manual para tener recibos observables. PE6 probó un refresco incremental manual, no una latencia garantizada de refresco automático. El nuevo entorno debe verificar ambas cosas por separado si se pretende afirmar comportamiento automático. Un consumidor atrasado puede alcanzar el stream si conserva offsets y la retención cubre el atraso; los registros ya expirados no reaparecen por aumentar RPU.

17. Materialized Views y consultas SQL

Las fuentes SQL numeradas01-07 son la autoridad; checkpoint_6_complete.sql es generado. El bootstrap aplica01-05, evitando recrear MV/rol existentes sin DROP. Las vistas analíticas son ordinarias y leen el estado actual de la MV raw. La vista histórica lakehouse consulta Iceberg en la misma sesión SQL.

```
SELECT DATE_TRUNC('minute',event_time) AS window_start, payment_method,
       COUNT(*) AS transaction_count, SUM(amount) AS total_amount,
       AVG(amount::decimal(24,6))::decimal(38,6) AS average_amount
FROM analytics.transactions_typed
WHERE validation_error IS NULL AND status='approved'
GROUP BY 1,2;
```

Se amplía la escala ANTES de AVG: convertir el resultado decimal de escala2 después del promedio no recupera precisión perdida. La corrección está revisada con documentación oficial y fixture decimal; falta ejecutarla en el nuevo Redshift. El JOIN usa inicio de ventana y método, no transaction_id contra un resumen. La nueva vista expone también arrival_timestamp para evidenciar frescura.

18. Consistencia de datos y reconciliación

At-most-once admite pérdidas al no reintentar; at-least-once admite repetición; exactly-once describe efectos coordinados bajo un protocolo y recuperación específicos. Idempotencia requiere que reintentar no cambie el efecto de negocio. El append actual no es idempotente por transaction_id. Iceberg ofrece visibilidad atómica por snapshot; las rutas independientes convergen eventualmente con latencias distintas y no comparten una transacción global.

Población del fixture nuevo	Cantidad	Total	Promedio
Aprobadas antes del cierre	3	30.01	10.003333
Aprobadas raw incluyendo tardía	4	1030.01	257.502500
Aprobadas equivalentes sin tardía confirmada	3	30.01	10.003333
Resultado AWS observado nuevo	Pendiente	Pendiente	Pendiente

Las diferencias esperadas no deben ocultarse. El tardío de1000 se publica solo después del primer commit y se excluye de Redshift únicamente si existe evidencia del descarte de Flink. El oráculo de esta prueba no generaliza a producción: haría falta un registro durable de elegibilidad/descarte por evento. Comparar métricas después de cierre y visibilidad, con run_id, minuto UTC, método y precisión decimal comunes. Un conteo bruto coincidente no prueba exactly-once.

19. Análisis de backpressure

Si el escritor Iceberg/S3 se ralentiza, se llenan buffers y el bloqueo se propaga aguas arriba por los operadores Flink. Puede crecer el atraso Kinesis y aumentar la duración/alineación de checkpoints hasta agotar timeout. Observar tasas, ocupación/backpressure, duración/fallos y progreso de commits antes de aumentar paralelismo: un cuello S3/Glue no se soluciona siempre con más tareas.

Redshift es un consumidor independiente. Sus consultas o refrescos lentos no envían backpressure directamente al job Flink. En consumo compartido sí existe competencia por capacidad de lectura de Kinesis. Por shard: entrada1 MB/s o 1000 registros/s; lectura compartida2 MB/s y hasta5 GetRecords/s.

Particiones sesgadas pueden saturar un shard aunque el promedio del stream sea bajo.

Mitigaciones: backoff con jitter en productores, claves distribuidas, batching, resharding o modo on-demand según patrón. Considerar EFO para consumidores compatibles y su costo, sin asumir que Redshift comparte esa API. Este análisis es documental; no se realizó un ensayo de saturación ni se afirma capacidad medida.

20. Manejo de fallos

Escenario	Señal y respuesta	Evidencia disponible
Shard saturado	Throttling, revisar clave/capacidad/backoff	Análisis; no carga AWS
Sink lento	Backpressure/CP largos, revisar S3 y archivos	Análisis
Fallo de tarea Flink	Restaurar checkpoint y reintentar	Inyección local pasada
Glue AccessDenied	Falla inicialización/commit; revisar rol y recursos	Análisis; no inyección
Commit concurrente	Conflicto/reintento optimista; revisar escritor duplicado	Sin fallo nuevo demostrado
S3 error/transitorio	Reintentos acotados; detener expansión del atraso	Análisis
Redshift atrasado	Revisar refresh, scan errors, capacidad y retención	PE6 refresco real; no estrés

Un commit con respuesta ambigua no habilita borrar metadatos a mano. Revisar el snapshot actual y el protocolo del sink antes de reintentar o reprocesar. Los archivos huérfanos tras fallos no deben eliminarse sin considerar lectores, retención y checkpoints. No se certifica cero errores futuros de concurrencia; solo puede declararse ausencia observada en un intervalo de logs identificado.

21. Observabilidad y monitoreo

Componente	Señales reales	Obtención / límites
Kinesis	IncomingRecords, IncomingBytes, GetRecords.IteratorAgeMilliseconds	AWS/Kinesis, StreamName
Kinesis	WriteProvisionedThroughputExceeded , ReadProvisionedThroughputExceeded	Revisar suma y períodos sin datos
Shards	Métricas por shard/clave caliente	Enhanced monitoring no habilitado por esta auditoría
Flink	numberOfCompletedCheckpoints, numberOfFailedCheckpoints, lastCheckpointDuration	AWS/KinesisAnalytics; confirmar series publicadas
Flink	backPressuredTimeMsPerSecond, numRecordsInPerSecond, numRecordsOutPerSecond	Dependen del nivel/dimensiones disponibles
Flink	currentInputWatermark, currentOutputWatermark, millisbehindLatest	Dashboard/REST/CloudWatch según operador/conector
Calidad	valid_records, invalid_records, late_records	Contadores del job; validar exposición administrada
Iceberg	snapshot actual, resumen de commit, metadata_location	Glue GetTable y JSON real S3
Redshift	SYS_STREAM_SCAN_STATES, SYS_STREAM_SCAN_ERRORS	SQL07; visibilidad según usuario
Redshift	SYS_MV_REFRESH_HISTORY, SYS_MV_STATE, MAX(llegada)	Refresco y frescura, no SLA automático

La configuración actual de Flink es APPLICATION. Las métricas de tarea/operador no se dan por disponibles en CloudWatch: usar Dashboard/REST o autorizar elevar el nivel y su costo. La documentación AWS restringe currentInputWatermark según dimensión/operador y marca backpressure en CloudWatch con una nota de versión1.13: para2.2 usar Dashboard/REST y comprobar disponibilidad, sin prometer esa serie administrada. El conector6.0 de Flink2.2 cambió métricas; no usar bytesRequestedPerFetch como si siguiera vigente.

SYS_STREAM_SCAN_STATES conserva posición por partición y tiempos de registros. MAX(arrival_timestamp) indica última llegada visible; sin tráfico nuevo, su edad crece sin que eso implique atraso. Combinar con eventos controlados y scan state. refresh_time no es instante de visibilidad final. Faltan capturas nuevas de Iterator Age, job activo y checkpoints; no se fabrican dashboards para suplirlas.

22. Seguridad e IAM

Principal	Acceso previsto	Restricciones / riesgo
Operador Terraform	Crear recursos/IAM e iniciar prueba	Perfil externo; nueva política pendiente revisión
Rol Flink	Leer stream, logs, escribir tabla S3/Glue	Prefijos/ARN del entorno; nuevo trust SourceAccount/Arn
Rol Redshift	Leer stream/KMS y Lakehouse S3/Glue	Sin escritura de negocio; condiciones KMS vía Kinesis
Rol SQL checkpoint6_analyst	Leer vistas agregadas/JOIN	Sin raw ni REFRESH; probar en nuevo entorno
Rol auditoría	Lectura de infraestructura/objetos definida	No confundir con administrador ni operador de deploy

Los comodines inevitables, como kinesis:ListStreams, se separan de permisos de datos por ARN. Glue necesita catálogo, base y tabla en operaciones jerárquicas. Redshift usa autenticación IAM entre servicios; el rol de servicio no es el rol SQL de analista. PE6 probó lectura permitida y raw/REFRESH denegados con un usuario de prueba, luego retirado. La identidad raíz usada para capturas no demuestra restricción analítica: esa evidencia procede del ensayo de permisos separado.

S3 tiene bloqueo público y SSE-S3; Kinesis usa clave administrada AWS; tráfico a servicios usa TLS. Redshift es privado. El trust endurecido de Flink se activa solo para el entorno nuevo, sin modificar políticas activas de dev. Falta probar esa asunción de rol en AWS. El perfil operador de PE6 no autoriza automáticamente nombres capstone; solicitar la ampliación concreta si el plan encuentra AccessDenied.

23. Optimización y costos

Modelo público revisado el07/11/2026 en us-east-1, sin consulta de factura. El archivo cost-model.json conserva cálculo Decimal y fuentes. Redshift4RPU a USD0.375/RPU-h implica USD1.50/h de actividad. Flink1KPU de proceso más1 de orquestación implica USD0.22/h. Kinesis2shards implica USD0.03/h; endpoint2AZ, USD0.02/h. Subtotal **USD1.77/h**, o USD3.54 por dos horas del supuesto de uso. No incluye almacenamiento/solicitudes/logs/transferencia/KMS/impuestos ni es tope.

En un entorno académico, mantener una sesión breve, fixture pequeño y retiro inmediato. Propuesta a aprobar: hasta dos horas, objetivo USD5 con contingencias, sin garantía de corte automático. Dev conserva un stream activo y objetos/versiones S3, logs y backend; no afirmar costo cero tras retirar solo Redshift.

En producción: medir carga por shard y lag antes de dimensionar; aumentar retención si el RTO real lo necesita; diseñar autoscaling y alarmas; compactar archivos y controlar snapshots con políticas probadas. Menor intervalo de checkpoint reduce frescura mínima potencial pero incrementa archivos, solicitudes y presión del catálogo. Mayor paralelismo consume KPUs y puede aumentar archivos sin resolver un cuello del catálogo. Batch puede ser más barato si la latencia requerida lo permite.

24. Pruebas end-to-end

Validación	Resultado observado	Alcance
Maven clean package	PASSED,10 pruebas	Java local; incluye minicluster/sink
Inspección JAR	PASSED	Clases/conectores/SDK; Flink provided excluido
Python unittest	PASSED,5 pruebas	Runner, fixture y dry run sin AWS
Terraform fmt/validate	PASSED,3 raíces	Sintaxis/esquema; warning backend hijo documentado
Plan dev refrescado	Solo output adicional	No apply ni cambios de recursos
E2E PE5/PE6	Evidencia histórica válida	Dos ensayos distintos, no clean deploy nuevo
Clean deploy capstone	Pendiente autorización	No confundir con validación local
Reconciliación simultánea nueva	Script preparado, no ejecutado	Resultados AWS pendientes

El script e2e verifica nombres/cuenta del manifiesto y exige MV inicial vacía. Espera Iceberg, confirma el descarte tardío en logs, refresca Redshift y compara COUNT/SUM/AVG por ventana. Conserva recibos y falla ante discrepancia; no imprime PASSED sin observaciones. La prueba no incluye carga, resharding, caída AWS ni garantía de auto-refresh. La prueba local de recuperación no sustituye esas ausencias.

25. Evidencias reales

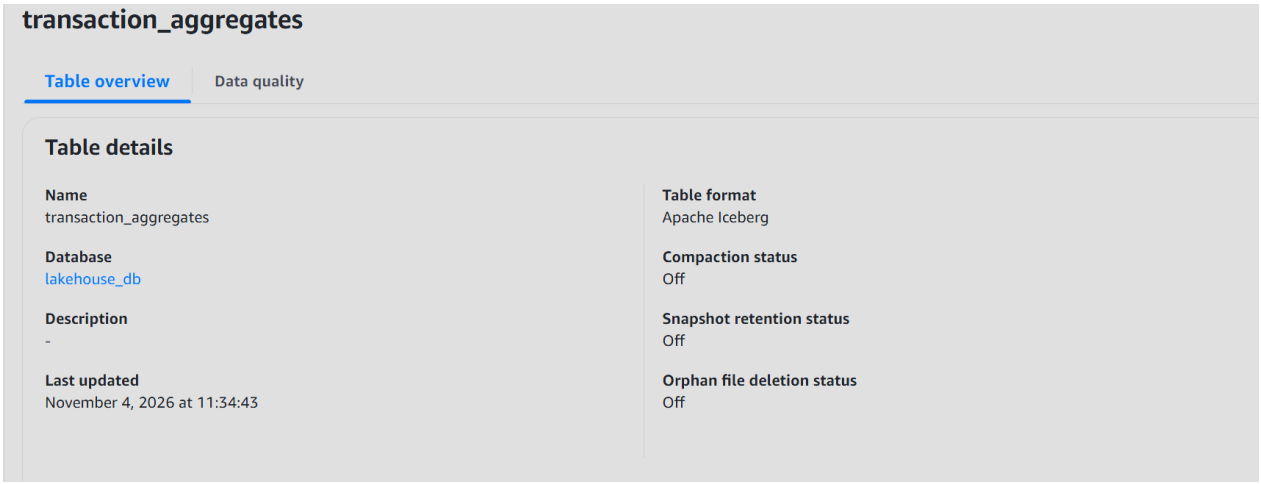
El PDF incorpora la captura Glue aportada en PE5 y las tres capturas reales de Query Editor de PE6. Cada una se identifica como antecedente. La fecha de última actualización mostrada por Glue no es necesariamente fecha de captura; si no está embebida, se declara desconocida. PE6 fue capturada durante la sesión del 07/11/2026 según los registros del ensayo. Las imágenes originales se conservan.

Los logs nuevos de Maven, inspección JAR, validaciones Terraform, plan e inventario se encuentran en docs/evidence/capstone. La inyección deliberada de fallo produce excepciones esperadas en el log local; el resultado final de tests es cero fallos. Faltan evidencias del nuevo entorno: apply desde cero, PutRecord, job RUNNING, CP, Iterator Age, metadatos nuevos y SQL reconciliado con llegada reciente. El runbook detalla cómo obtenerlas y registrar fecha/interpretación.

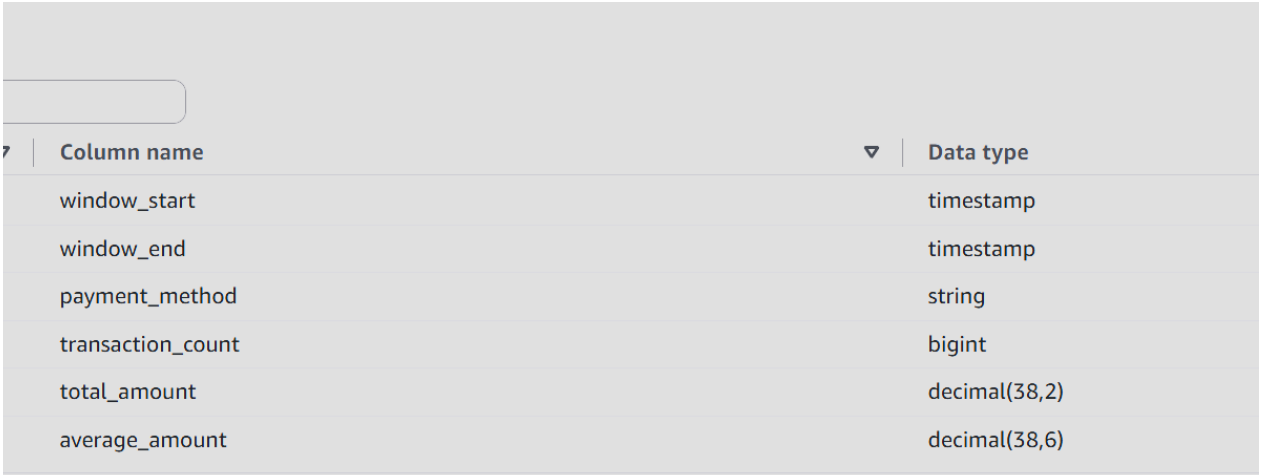
Evidencia histórica · Glue: tabla Iceberg

AWS Glue Data Catalog. Captura aportada para PE5; fecha exacta de captura no embebida. La consola muestra actualización del 04/11/2026. Revisada el 07/11/2026. Confirma nombre/base, formato Iceberg y seis columnas; no prueba datos del fixture capstone.

Tabla, base y formato Iceberg



Esquema: nombres y tipos de las seis columnas



Column name	Data type
window_start	timestamp
window_end	timestamp
payment_method	string
transaction_count	bigint
total_amount	decimal(38,2)
average_amount	decimal(38,6)

Evidencia histórica · Redshift: eventos recibidos

Query Editor v2, sesión PE6 del 07/11/2026. Seis registros válidos, con aprobados y rechazados. Son eventos del ensayo histórico; no corresponden al nuevo despliegue.

SQL ejecutado

```sql
SELECT transaction_id, event_time, amount, payment_method, status
FROM analytics.transactions_typed
WHERE validation_error IS NULL
ORDER BY payment_method, event_time;
```

Seis eventos válidos

Result 1 (6)

transaction_id	event_time	amount	payment_method	status
pe6_20261107_033721-1-0	2026-11-04 11:33:01	10.1	card_pe5_20261104_113…	approved
pe6_20261107_033721-1-1	2026-11-04 11:33:02	20.2	card_pe5_20261104_113…	approved
pe6_20261107_033721-1-2	2026-11-04 11:33:03	999	card_pe5_20261104_113…	rejected
pe6_20261107_033721-0-0	2026-11-07 03:37:01	10.1	card_pe6_20261107_033…	approved
pe6_20261107_033721-0-1	2026-11-07 03:37:02	20.2	card_pe6_20261107_033…	approved
pe6_20261107_033721-0-2	2026-11-07 03:37:03	999	card_pe6_20261107_033…	rejected

Evidencia histórica · Redshift: métricas por minuto

Query Editor v2, sesión PE6 del 07/11/2026. Dos grupos, tres transacciones y dos aprobadas por grupo; importe aprobado 30.30. El promedio de este lote no detectaba el problema de escala corregido.

SQL ejecutado

```
SELECT *
FROM analytics.transactions_per_minute
ORDER BY window_start DESC;
```

Identificación y totales

window_start	payment_method	transaction_count	total_amount
2026-11-07 03:37:00	card_pe6_20261107_033…	3	1029.3
2026-11-04 11:33:00	card_pe5_20261104_113…	3	1029.3

Promedio, estados e importe aprobado

average_amount	approved_count	rejected_count	approved_amount
343.1	2	1	30.3
343.1	2	1	30.3

Evidencia histórica · Redshift: JOIN con histórico

Query Editor v2, sesión PE6 del 07/11/2026. Una fila concilia 2 aprobadas y 30.30 contra el snapshot PE5. Flink estaba detenido; esto prueba consulta conjunta, no actualización simultánea de ambas rutas.

SQL ejecutado

```
1  SELECT *
2  FROM analytics.recent_vs_iceberg;
```

Ventana, método y cantidades

window_start	payment_method	recent_approved_co…	iceberg_approved_c…
2026-11-04 11:33:00	card_pe5_20261104_113…	2	2

Importes y diferencias

recent_approved_am…	iceberg_approved_a…	count_difference	amount_difference
30.3	30.3	0	0

26. Análisis de trade-offs

Decisión	Beneficio	Costo / alternativa
Dos consumidores independientes	Analítica raw e histórico transformado	Consistencia eventual; reconciliación explícita
Ventanas1min/desorden10s	Agregación simple y latencia contenida	Tardíos no corrigen; side output requiere retención durable
Append sin deduplicación	Estado y operación simples	Duplicados de negocio cuentan; dedup añade estado/TTL
Partición diaria	Poda temporal y menos particiones	Consultas puntuales aún leen archivos del día
Redshift Serverless	Sin gestión de nodos	Refresco/streaming pueden sostener cómputo
Sin NAT/IGW en VPC	Menor superficie/costo NAT	Endpoints y compatibilidad de servicio necesarios
Backend aislado	No arriesga estado dev	Bootstrap y retiro separados

27. Limitaciones y riesgos residuales

Bloqueante para certificar entrega: falta ejecutar la ruta limpia, verificar permisos del entorno nuevo y capturar reconciliación/observabilidad actuales. El flujo SQL y trust nuevo validan localmente pero no han sido ejercitados en AWS. Las cuotas regionales, propagación IAM y compatibilidad efectiva se verifican durante esa sesión; cualquier error debe conservarse como evidencia, no ocultarse.

No hay deduplicación por ID, ledger durable de tardíos, SLO medido, saturación probada ni recuperación multirregión. El plan con backend S3 nuevo aún no se ejecutó porque ese backend no fue creado; sí existe una previsualización desde estado local vacío que no debe aplicarse. La limpieza requiere una excepción explícita y acotada para la base Glue protegida. El JAR1.2 no se desplegó sobre dev y su compatibilidad de snapshots con1.1 no se afirma. El escaneo de secretos por patrones no reemplaza una auditoría completa del historial.

28. Conclusiones

La evidencia demuestra piezas funcionales de Lakehouse e ingesta analítica y pruebas locales de semántica/recuperación. La auditoría hace verificable la diferencia entre esos antecedentes y un sistema reproducible desde cero. El repositorio ahora prepara esa validación con nombres, estado, artefacto y fixture separados, sin destruir ni reiniciar la infraestructura anterior.

El resultado académico final todavía está pendiente. Solo después del apply aislado, las comprobaciones E2E, las capturas nuevas y el retiro autorizado podrá quitarse la condición preliminar del PDF. No existe evidencia suficiente para afirmar exactly-once global entre Redshift y Flink/Iceberg.

29. Referencias técnicas

Documentación oficial consultada para decisiones; no implica validación del entorno:

• AWS: métricas Managed Flink.

• Apache Flink2.2: ventanas.

• Apache Iceberg: integración AWS.

• Apache Iceberg: sink Flink y checkpoints.

• AWS: AVG y escala decimal.

• AWS: SYS_STREAM_SCAN_STATES.

• AWS: GetRecords y límites.

• Precios Redshift.

• Precios Flink.

• Precios Kinesis.

• Precios PrivateLink.

30. Enlace al repositorio GitHub

Repositorio Coderhouse Data Platform.

Rama de auditoría final.

La rama debe revisarse mediante Pull Request hacia main, sin merge automático. La entrega6 es la base del trabajo y no se presume fusionada. El PDF es el entregable académico; el enlace permite inspeccionar fuentes, pruebas y evidencias.